

ECE 486: Final Project Report

Spring 2024

Final Project Report

TA: Yu-Ju Chang, Section B

Dash Kamriani Beard, Aniketh Rayudu

Introduction

We created a controller that inverts a pendulum for our ECE 486 final project. Our system is a pendulum made up of a metal arm, a reaction wheel rotor connected to the end of the pendulum, a motor attached to the reaction wheel that allows us to control how fast we spin it, and two sensors that can measure the angle of the rotor and the angle of the pendulum. Altogether, our system is called a reaction-wheel pendulum (RWP). The goal of our controller is to take our system at stable equilibrium with the pendulum facing down, swing the pendulum by rotating the rotor in different directions, and stabilize the pendulum when it is inverted at unstable equilibrium. The spinning of the rotor spins our pendulum arm due to Newton's Third Law: when we spin our rotor, we are applying a torque to the rotor. Because the rotor is attached to the free-swinging pendulum, the rotor applies an equal and opposite torque to the pendulum arm, which we can use to swing our pendulum. We can then design a controller using the sensors in our system. We must first create a mathematical model of our system: model all the variables using a system of 2nd order differential equations, plug in things we do know, and then solve for things we don't know. To create our model in practice, we used the Lagrange method. We then created several iterations of negative feedback controllers to test, improve, and practice using different methods of control. This report will detail our process of making and refining a system model, as well as iteratively designing controllers to stabilize our system.

Mathematical Model

$$\theta_p'' + \omega_{np}^2 \sin \theta_p = -\frac{k}{J} i$$

$$\theta_r'' = \frac{k}{J} i$$

This set of equations illustrates how the torque on the rotor generated by the current will change the angular acceleration of θ_p and θ_r by a transducer factor J and k.

The linearized (at $\theta_p = \pi$) state space model is as follows:

$$\begin{pmatrix} \delta \theta_p' \\ \theta_p'' \\ \delta \theta_r' \\ \theta_r'' \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ a & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} * \begin{pmatrix} \delta \theta_p \\ \theta_p' \\ \delta \theta_r \\ \theta_r' \end{pmatrix} + \begin{pmatrix} 0 \\ -b_p \\ 0 \\ b_r \end{pmatrix} * u$$

$$\text{Where } \delta \theta_p = \theta_p - \pi, a = \omega_{np}^2, b_p = \frac{k_u}{J}, b_r = \frac{k_u}{J_r}$$

Developing PD Control

We began designing our feedback controllers by identifying key elements of our system. We determined the static and viscous friction by making our motor run at a few constant reference speeds,

measuring the amount of control needed, and fitting the data to get values for each type of friction. This works because our model of wheel spinning at steady state ($\dot{\omega}_r = \ddot{\theta}_r = b_r (u + F(\dot{\theta}_r)) = 0$) has a control effort that is equal to the level of friction that the wheel has. Thus, we can just collect this data and invert it to find the force of friction. We chose to neglect friction in the pendulum arm, as it uses a bearing to spin, which we believe is efficient and results in negligible friction. We then analyzed the natural frequency of the pendulum by lifting it and measuring the frequency at which it swings. Then, we had to determine the torque constant of the motor by solving for b_r and b_p in our system. A naïve approach is to measure θ_p and θ_r , differentiate our values twice to get all the necessary values, and plug and chug to solve for b_r and b_p . Unfortunately, our angular data is very noisy and results in increasingly meaningless first and second derivatives. Instead, we can find a polynomial fit to our data to get the smoothness we need, then differentiate those values to find our torque constants. In practice, we used a cubic fit to obtain the necessary differentiability and used the given values of $b_r = 198$ and $b_p = 1.08$ because they were close enough to what we found.

After system identification, we began designing controllers to stabilize our system. We first designed a 2-state feedback system, where we just controlled the pendulum angle. We then added control for the wheel velocity so that the control would be limited and converge to a steady state, which made our controller a 3-state feedback system. Also added friction compensation, which is just $b\omega + c$ to the signal after the controller. Friction compensation was beneficial: it made our system converge to steady state faster. We think this is because, with friction compensation, the controller was able to deal with exclusively the non-friction forces, which allowed for finer tuning, instead of having to combat friction too.

Equilibrium

$$x = \begin{pmatrix} \delta\theta_p \\ \theta'_p \\ \delta\theta_r \\ \theta'_r \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \delta\theta_r \\ \theta'_r \end{pmatrix}, u = 0 \rightarrow x' = \begin{pmatrix} 0 & 1 & 0 & 0 \\ a & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} * x = \begin{pmatrix} 0 \\ 0 \\ \theta'_r \\ 0 \end{pmatrix}$$

Thus, the system is stable at equilibrium point. Since full state feedback is adopted,

$$u = -Kx$$

$$x' = Ax - BKx$$

$$= x' = (A - BK)x$$

Since the three states we care about are pendulum angle, pendulum speed, and rotor speed, our goal is to make $\delta\theta_p, \theta'_p, \theta'_r$ converge to 0.

```

bp_max = 0.95671;
br_max = 172.0067;

A = [0,1,0,0; a,0,0,0; 0,0,0,1; 0,0,0,0];
B = [0; -bp_max; 0; br_max];
p = [-5.28+7.04i, -5.28-7.04i, 0, -5.28];
K = place(A,B,p);

```

Output:

```
> K = [-218.824, -22.1688, 0, -0.0312]
```

This K value will make $\delta\theta_p$ converge to 0, and thus θ_p converge to π .

	Two-State Feedback		Three-State Feedback		Observer	
Max IC deviations	$0.1225 \delta\theta_p$	$0.9 \delta\dot{\theta}_p$	$0.13 \delta\theta_p$	$0.95 \delta\dot{\theta}_p$	$0.0637 \delta\theta_p$	$0.55 \delta\dot{\theta}_p$
Max pulse	4		6		3.7	
Max disturbance	4		7		4.5	

Table 1

Under the two-state feedback, the model runs fine with only initial conditions, but for max pulse and max disturbance, the motor undergoes a constant acceleration to counteract, however small the disturbance is.

Under three state feedback, a pole is placed in LHS for θ'_r so that it will converge to 0 too, and thus, the angular velocity will not go to infinity as with two state feedback.

Also, the original code only works so that the pendulum will only stabilize when rotated CCW. This is because the controller is trying to stabilize at π . With the current implementation, π is at a half CCW turn. Another turn after that would be at $3\pi/2$. Then the controller would always want to spin CW to make the pendulum at π . Instead, we took the angle signal and modded it by 2π . This way the max angle is 2π , and the pendulum will always try to stabilize at π whether we spin the pendulum CW or CCW.

Full State Feedback Control with Decoupled Observer (2-3 pages, 50 pts)

Observers are used when not all states are obtainable, but it can estimate the state based on the input and observed output. The reason why the observer can decouple is that $\delta\theta_p, \theta'_p$ are independent from $\delta\theta_r, \theta'_r$, based on the state space equation above. Also, some entries in L does not affect the pole placement, but it amplifies the noises and errors in practice, thus the opposite of which becomes the advantage of the two-state observer.

- 2. Let's show that the error between the actual states and estimated states

$$\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$$

goes to zero over time.

First we differentiate the equation for the error. Next, prove that the poles are stable.

Set $\mathbf{e}_{\text{dot}}$ equal to zero and show that $\mathbf{e} = [0,0,0,0]$ is the *only* stable equilibrium for this equation.

$$\mathbf{e}' = \mathbf{x}' - \mathbf{x}^{\wedge}'$$

Next we substitute the equations we have for $\mathbf{x}_{\text{dot}}$, $\mathbf{xhat}_{\text{dot}}$, and u . $\mathbf{e}$ is the only variable.

$$\mathbf{e}' = A\mathbf{x} + B\mathbf{u} - (A\mathbf{x}^{\wedge} + B\mathbf{u} + L(\mathbf{y} - C\mathbf{x}^{\wedge}))$$

$$\mathbf{e}' = A(\mathbf{x} - \mathbf{x}^{\wedge}) - LC * (\mathbf{x} - \mathbf{x}^{\wedge})$$

$$\mathbf{e}' = (A - LC) * (\mathbf{x} - \mathbf{x}^{\wedge})$$

$$\mathbf{e}' = (A - LC) * \mathbf{e}$$

If $\mathbf{e}'=0$ and we know $A-LC$ is stable and full rank, $\mathbf{e} = [0,0,0,0]$. Since the goal is to make $\mathbf{x} - \mathbf{x}^{\wedge}$ converge to 0 much faster than the system itself does, the poles for $(A-LC)$ needs to be set much further in LHS than the original system.

% eigenvalues of A-LC are poles of system

```
A = [0,1,0,0; a,0,0,0; 0,0,0,1; 0,0,0,0];
B = [0; -bp_max; 0; br_max];
C = [1 0 0 0; 0 0 1 0];
D = zeros(4,3)
```

```
wn = 10;
zeta = 0.5;
```

```
sys = tf([wn^2], [1 2*zeta*wn wn^2]);
poles_new = pole(sys);
```

```
K_new = place(A, B, [poles_new(1) poles_new(2) 0 sum(poles_new)/2]);
```

```
A12 = [0 1; a 0];
B12 = [0; -bp_max];
C12 = [1 0];
```

```
p12 = [(-5*30)+7.04i, (-5*30)-7.04i];
% p12 = [real(poles_new(1))*7 + imag(poles_new(1))*j/30, real(poles_new(1))*7 +
imag(poles_new(2))*j/30];
L12 = place(A12', C12', p12)';
```

```
A34 = [0 1; 0 0];
```

```

B34 = [0; br_max];
C34 = [1 0];
p34 = [-50, -40];
% p34 = [sum(poles_new)/2*6, sum(poles_new)/2*7];
L34 = place(A34', C34', p34)';

L_new = [L12, [0; 0]; [0; 0], L34];
% C_new = [C12, 0, 0; 0, 0, C34]
% D_new = zeros(2,3)
eig(A-L_new*C)
% eig(A)
% eig(A12)
% eig(A34)

```

Verification:
`eig(A-L_new*C):`

ans =

```

1.0e+02 *

-1.5000 + 0.0704i

-1.5000 - 0.0704i

-0.5000 + 0.0000i

-0.4000 + 0.0000i

```

- 3.

Values listed in Table 1 show the maximum IC deviation, pulse disturbance, and constant perturbation that the controller can stabilize from the Simulink simulations of the RWP.

- 4.

The behavior of the controller with the observer tends to be more sensitive as it can only handle a smaller range of IC, max pulse, and max disturbance compared to full state feedback controller. It does make sense because the all the state are being estimated instead of being directly observed.

Conclusions (1 paragraph of text, 10 pts)

Judging by range of IC deviation, max pulse, and max disturbance, full state feedback controller is better as it can take in a larger range of them and still can bring the system back to stability. On the other hand, if the states are not easily obtainable, the observer definitely wins out since there is no way to implement full state feedback controller.

Part 6:

Part 6 of our project required us to use a different controller based on the position of the pendulum. If the pendulum is placed at the upward position, it should stabilize upward, and if it is placed downward, it should stabilize downward. We re-implemented our previous state-feedback controller by adding two switches with conditions for each case. We decided to detect when the pendulum is facing up or down by taking the cosine of the $\text{mod}(2\pi)$ pendulum angle so that our switch gives us an “up controller” when the angle is positive and a “down controller” when it is negative.

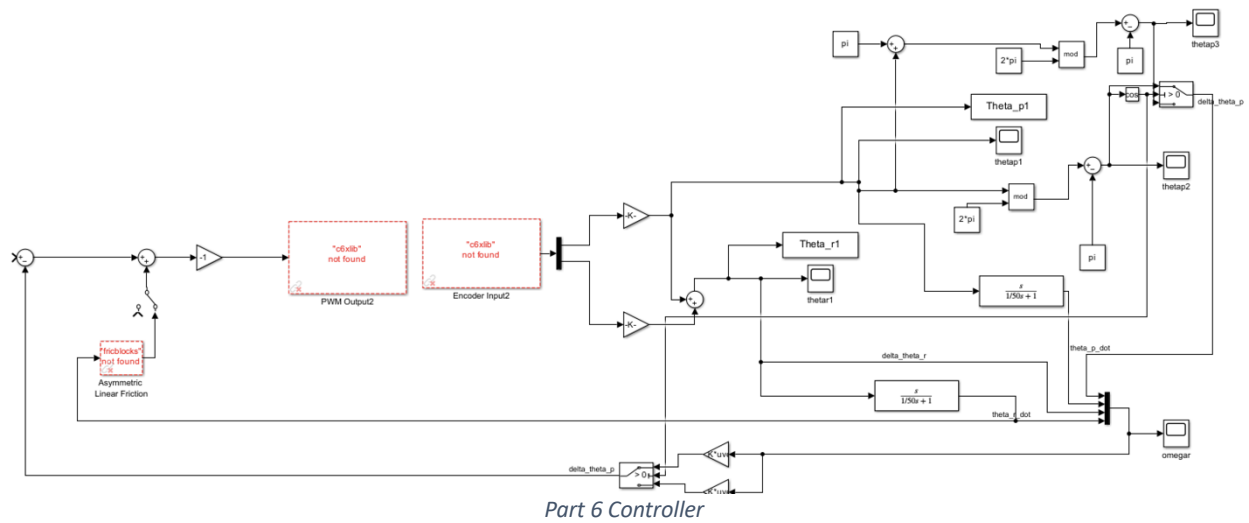
Our up controller has the same behavior as the normal inverted stabilizing controller we already had, so we didn’t need to make anything new. However, to stabilize down, we had to design a down controller. For this new case, we had to change the reference angle of the controller to 0 rad instead of π , and we also had to re-linearize our system to make the controller spin the correct way to stabilize at the bottom. This only involved adding a negative sign in our A matrix to the linearly approximated term. Our end behavior resulted in the controller stabilizing to the down position if it was held below the horizontal axis and stabilizing to the inverted position if held above the horizontal axis.

Part 7:

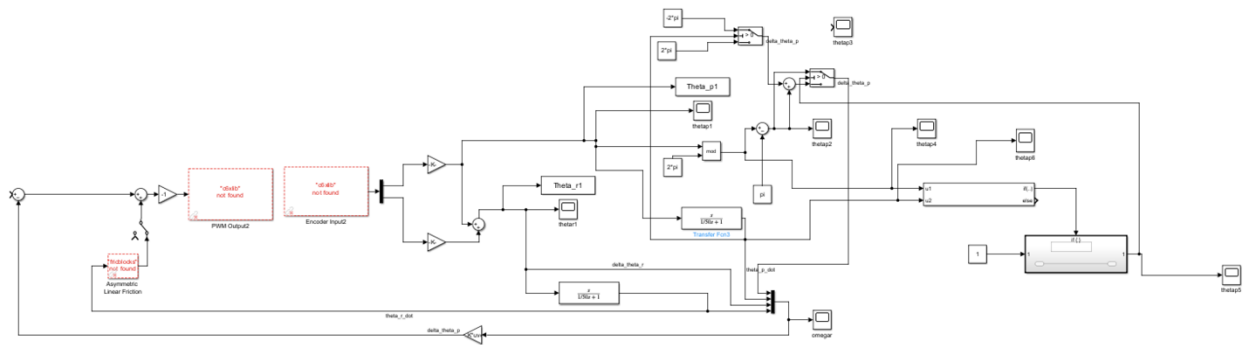
Part 7 of the project required us to make a swing-up controller. We had previously made a controller that does this behavior in part 4, but it was very slow. The reason why our initial part 4 controller worked at all was because of the strength of the motor. In simple terms, the way that our part 4 controller worked was it calculated the shortest distance towards the upward position, and then tried to always move the pendulum in that direction. So, if our pendulum was on the right side of the setup, it would always try to spin up to the right, and vice versa for the left side. If our motor was able to spin the reaction wheel fast enough to counteract gravity, it would have worked in one swing, but our motor was not that strong. Therefore, what happened was our controller would spin the rotor a little bit in one direction, gravity would then overpower it despite the motor trying to go the other way, and the pendulum would then gain a little momentum towards the opposite direction on the down swing while still spinning the rotor the wrong way. Then at the very bottom, it would switch rotor spin direction because the pathway now became shorter in the opposite direction, and this process would continue until eventually, after around 50 seconds, the pendulum would swing up. This is obviously very inefficient, so we decided to develop a new method.

After a few rounds of testing, the method we settled on was this: spin the rotor one way, detect when the pendulum stops moving ($\dot{\theta}_p$ becomes 0 and swaps sign), and then spin the rotor the other way at the top of the swing. We implemented this by creating an if statement subroutine that detects which direction the wheel is spinning per side, detects which way it should be spinning based on the sign of $\dot{\theta}_p$, and then spins it properly. Initially, we used the down controller from part 6 to make it spin downwards at the top of the swing, but we realized that this wasn’t as efficient as possible because the down controller was “trying” to stabilize at the bottom, not swing down so that the pendulum can gain momentum up to the top. We solved this by doing a coordinate transformation: using the property that our controller will always try to take the shortest path towards the top, we added or subtracted 2π based on our current position when $\dot{\theta}_p$ changes sign. If it’s on the left, we will subtract 2π to turn our current coordinate from $\sim\pi/4$ to $(-2\pi + \sim\pi/4)$, vice versa on the right side, which makes our controller try to swing up the opposite direction because that is now the shortest distance to π . This method worked extremely well compared to our old method, and it made our swing-up controller fully stabilize on average in 9 swings, down from nearly 80.

Appendix



Part 6 Controller



Part 7 Controller